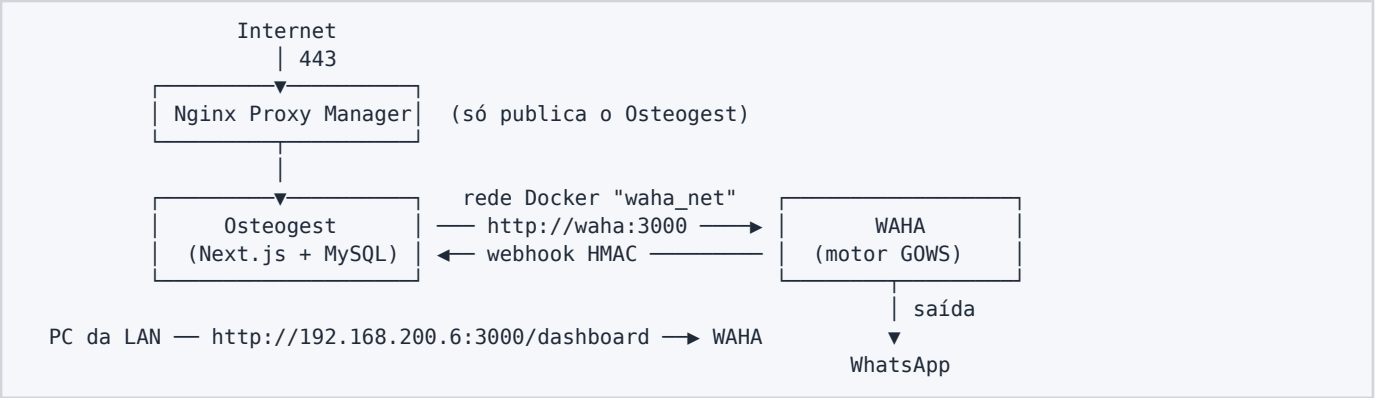
Guia de Deploy — WAHA no Portainer + Nginx Proxy Manager

Projeto: integração WhatsApp do Osteogest
Repositório: https://github.com/duquedaniel/WAHA (branch `main`)
Servidores: demo `192.168.200.6` (demo.osteogest.ddx.pt) · produção `vilma.osteogest.ddx.pt`

1. Visão geral

O WAHA (WhatsApp HTTP API) corre como uma stack independente no Portainer e **só é acessível dentro da rede local** do servidor. O Osteogest fala com o WAHA através de uma rede Docker interna partilhada (`waha_net`) e o WAHA devolve eventos (estado da sessão, entregas, respostas) ao Osteogest por webhooks assinados com HMAC.



Princípios de segurança:

- A porta do WAHA é publicada **apenas no IP da LAN** (nunca em `0.0.0.0`).
- **Não existe proxy host público** para o WAHA no Nginx Proxy Manager.
- O router **não** faz port-forward da porta 3000.
- Todas as chamadas à API exigem `X-Api-Key`; todos os webhooks são assinados (HMAC-SHA512).

2. Pré-requisitos

- Portainer (Docker standalone) com acesso ao servidor.
- Stack do Osteogest já existente no mesmo servidor.
- Nginx Proxy Manager (NPM) a servir o Osteogest.
- Um número de WhatsApp dedicado à clínica (pode ser WhatsApp Business no telemóvel; o WAHA liga-se como "dispositivo ligado").
- Acesso SSH ao servidor (para gerar segredos e validações).

3. Gerar os segredos

No servidor (ou em qualquer máquina com OpenSSL), gerar **três** valores diferentes:

```
openssl rand -hex 32   # WAHA_API_KEY
openssl rand -hex 32   # WAHA_DASHBOARD_PASSWORD
openssl rand -hex 32   # WAHA_HOOK_HMAC_KEY
```

> Guarde-os num gestor de passwords. Use segredos **diferentes** em demo e em produção. A

> `WAHA_API_KEY` e a `WAHA_HOOK_HMAC_KEY` terão de ser configuradas também no Osteogest.

4. Criar a stack do WAHA no Portainer

4.1 Método recomendado — Repository

1. Portainer → **Stacks** → **Add stack**.
2. **Name:** `waha`
3. **Build method:** `Repository`
4. **Repository URL:** `https://github.com/duquedaniel/WAHA`
5. **Repository reference:** `refs/heads/main`
6. **Compose path:** `deploy/portainer/docker-compose.yml`
7. Se o repositório for privado: ativar **Authentication** e usar o utilizador GitHub + um Personal Access Token (fine-grained, permissão Contents: Read-only apenas neste repositório).
8. (Opcional) **GitOps updates** desligado — atualizações controladas manualmente.
9. Preencher as **Environment variables** (secção 4.3).
10. **Deploy the stack**.

4.2 Método alternativo — Web editor

Se preferir não ligar o Portainer ao GitHub, escolha **Web editor** e cole o conteúdo abaixo (igual ao ficheiro `deploy/portainer/docker-compose.yml`):

```
name: waha

services:
  waha:
    image: devlikeapro/waha:${WAHA_IMAGE_TAG:-gows}
    container_name: waha
    restart: unless-stopped
    dns:
      - 1.1.1.1
      - 8.8.8.8
    # Published only on the server LAN IP, never on 0.0.0.0
    ports:
      - '${WAHA_LAN_IP:?define WAHA_LAN_IP}:${WAHA_LAN_PORT:-3000}:3000/tcp'
    networks:
      - waha_net
    volumes:
      - waha_sessions:/app/.sessions
      - waha_media:/app/.media
    environment:
      TZ: Europe/Lisbon
      WHATSAPP_DEFAULT_ENGINE: GOWS
      WAHA_BASE_URL: http://waha:3000
      WAHA_LOG_LEVEL: info
      WAHA_PRINT_QR: 'false'
      WHATSAPP_RESTART_ALL_SESSIONS: 'True'

      WAHA_API_KEY: ${WAHA_API_KEY:?define WAHA_API_KEY}
      WAHA_DASHBOARD_ENABLED: 'true'
      WAHA_DASHBOARD_USERNAME: ${WAHA_DASHBOARD_USERNAME:-admin}
      WAHA_DASHBOARD_PASSWORD: ${WAHA_DASHBOARD_PASSWORD:?define WAHA_DASHBOARD_PASSWORD}
      WHATSAPP_SWAGGER_ENABLED: 'true'
      WHATSAPP_SWAGGER_USERNAME: ${WAHA_DASHBOARD_USERNAME:-admin}
      WHATSAPP_SWAGGER_PASSWORD: ${WAHA_DASHBOARD_PASSWORD:?define WAHA_DASHBOARD_PASSWORD}

      WHATSAPP_HOOK_URL: ${OSTEOGEST_WEBHOOK_URL:-}
      WHATSAPP_HOOK_EVENTS: session.status,message,message.ack
```

```
      WHATSAPP_HOOK_HMAC_KEY: ${WAHA_HOOK_HMAC_KEY:?define WAHA_HOOK_HMAC_KEY}
      WHATSAPP_HOOK_RETRIES_POLICY: exponential
      WHATSAPP_HOOK_RETRIES_DELAY_SECONDS: '2'
      WHATSAPP_HOOK_RETRIES_ATTEMPTS: '10'

      WAHA_MEDIA_STORAGE: LOCAL
      WHATSAPP_FILES_FOLDER: /app/.media
    healthcheck:
      test: ['CMD', 'curl', '-fsS', 'http://localhost:3000/ping']
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 30s
    logging:
      driver: json-file
      options:
        max-size: 50m
        max-file: '5'

networks:
  # Shared with the Osteogest stack (declare it there as external)
  waha_net:
    name: waha_net

volumes:
  waha_sessions:
  waha_media:
```

4.3 Variáveis de ambiente da stack

Na secção **Environment variables** use **Advanced mode** e cole (ajustando os valores), ou **Load variables from .env file** com `deploy/portainer/stack.env.example` preenchido:

```
WAHA_LAN_IP=192.168.200.6
WAHA_LAN_PORT=3000
WAHA_IMAGE_TAG=gows
WAHA_API_KEY=<segredo 1>
WAHA_DASHBOARD_USERNAME=admin
WAHA_DASHBOARD_PASSWORD=<segredo 2>
WAHA_HOOK_HMAC_KEY=<segredo 3>
OSTEOGEST_WEBHOOK_URL=http://<servico-osteogest>:<porta-interna>/api/webhooks/waha
```

Variável	Descrição	Demo	Produção
`WAHA_LAN_IP`	IP da LAN onde a porta é publicada	`192.168.200.6`	IP LAN do servidor vilma
`WAHA_LAN_PORT`	Porta no host	`3000`	`3000`
`WAHA_IMAGE_TAG`	Motor/arquitetura da imagem	`gows`	`gows` (ou versão fixa)
`WAHA_API_KEY`	Chave da API (header `X-Api-Key`)	segredo 1	segredo novo
`WAHA_DASHBOARD_PASSWORD`	Password do dashboard e Swagger	segredo 2	segredo novo
`WAHA_HOOK_HMAC_KEY`	Chave de assinatura dos webhooks	segredo 3	segredo novo
`OSTEOGEST_WEBHOOK_URL`	Endpoint do Osteogest na rede interna	ver nota	ver nota

Nota sobre `OSTEOGEST_WEBHOOK_URL`: use o **nome do serviço** (ou `container_name`) do Osteogest no docker-

compose dele e a **porta interna** do Next.js (normalmente `3000`), por exemplo `http://osteogest:3000/api/webhooks/waha`. Enquanto o módulo WhatsApp do Osteogest não existir, pode deixar vazio — o WAHA simplesmente não envia webhooks.

Imagens disponíveis (`WAHA_IMAGE_TAG`): `gows` (recomendado, sem browser), `noweb` (sem browser), `latest` (WEBJS com Chromium, mais pesado); em servidores ARM: `gows-arm`, `noweb-arm`, `arm`. Em produção pode fixar uma versão (ex.: `gows-2026.9.1`) para evitar atualizações inesperadas.

5. Ligar o Osteogest à rede `waha_net`

A stack do WAHA cria a rede `waha_net`. **Faça primeiro o deploy do WAHA** e só depois edite a stack do Osteogest, acrescentando a rede externa ao serviço da aplicação (não à base de dados):

```
services:
  osteogest:              # nome real do serviço da app Next.js
    # ... configuração existente ...
    environment:
      # ... variáveis existentes ...
      WAHA_BASE_URL: http://waha:3000
      WAHA_API_KEY: ${WAHA_API_KEY}
      WAHA_WEBHOOK_HMAC_KEY: ${WAHA_HOOK_HMAC_KEY}
    networks:
      - default           # manter a rede atual (MySQL, NPM, ...)
      - waha_net

networks:
  default: {}             # se já existir uma rede nomeada, mantenha-a
  waha_net:
    external: true
```

- Se o serviço já tiver uma lista `networks:`, **acrescente** apenas `waha_net` — não remova as existentes, senão o Osteogest perde o acesso ao MySQL e ao NPM.
- Adicione `WAHA_API_KEY` e `WAHA_HOOK_HMAC_KEY` às Environment variables da stack do Osteogest com **os mesmos valores** da stack do WAHA.
- **Update the stack** no Portainer.

6. Validar a instalação

6.1 No servidor (SSH)

```
# a porta deve estar ligada APENAS ao IP da LAN
ss -ltnp | grep 3000
# esperado: 192.168.200.6:3000  (NÃO deve aparecer 0.0.0.0:3000 nem [::]:3000)

# estado do container
docker ps --filter name=waha --format '{{.Names}}  {{.Status}}'
# esperado: waha  Up ... (healthy)

# logs
docker logs --tail 50 waha
```

6.2 A partir de um PC da rede local

- `http://192.168.200.6:3000/ping` → `{"message":"pong"}`
- `http://192.168.200.6:3000/dashboard` → pede utilizador/password (admin + segredo 2)
- `http://192.168.200.6:3000/` → Swagger (documentação da API)

6.3 A partir do container do Osteogest (rede interna)

```
docker exec -it <container-osteogest> node -e "fetch('http://waha:3000/api/sessions',{headers:{'X-Api-Key':process.env.WAHA_API_KEY}}).then(r=>r.text()).then(console.log)"
# esperado: []  (lista de sessões, vazia no início)
```

6.4 A partir da internet (telemóvel em 4G)

- `http://<IP público>:3000/ping` → **tem de falhar** (timeout).
- Nenhum domínio público deve abrir o WAHA.

7. Ligar o número WhatsApp (primeira vez)

Quando o módulo WhatsApp do Osteogest estiver pronto, o emparelhamento faz-se a partir do próprio Osteogest. Até lá, ou para testes, pode fazê-lo no dashboard do WAHA:

1. Abrir `http://192.168.200.6:3000/dashboard` e autenticar.
2. Adicionar o servidor: URL `http://192.168.200.6:3000`, API key = `WAHA_API_KEY`.
3. **Start New** sessão com o nome **`osteogest`** (é o nome que o Osteogest vai usar).
4. Escolher **QR code** ou **código de emparelhamento** (número em formato internacional, ex.: `351912345678`).
5. No telemóvel da clínica: WhatsApp → **Definições → Dispositivos ligados → Ligar um dispositivo** (ou Ligar com número de telefone para o código).
6. O estado deve passar a **WORKING**.

Teste rápido de envio (a partir de um PC da LAN):

```
curl -s -X POST http://192.168.200.6:3000/api/sendText \
  -H "X-Api-Key: <WAHA_API_KEY>" -H "Content-Type: application/json" \
  -d '{"session":"osteogest","chatId":"351910000000@c.us","text":"Teste WAHA"}'
```

8. Configuração do Nginx Proxy Manager

8.1 WAHA — recomendação: nenhum proxy host

A forma mais segura é **não criar** qualquer proxy host para o WAHA. O acesso administrativo faz-se diretamente por `http://192.168.200.6:3000` a partir da LAN, e o Osteogest usa `http://waha:3000` pela rede Docker. Nada do WAHA fica exposto no NPM.

8.2 WAHA — opcional: proxy host restrito à LAN (com HTTPS)

Só se quiser um endereço com nome e HTTPS dentro da LAN (ex.: `waha.lan.ddx.pt`).

Passo 1 — Access List (NPM → Access Lists → Add Access List):

Campo	Valor
Name	`LAN-only`
Satisfy Any	desligado
Pass Auth to Host	desligado
Authorization	(vazio — o WAHA já tem autenticação própria)
Access → allow	`192.168.200.0/24` (ajuste à sua sub-rede)

Campo	Valor
Access → allow	`172.16.0.0/12` (redes Docker, opcional)
Access → deny	`all`

Passo 2 — Proxy Host (NPM → Hosts → Proxy Hosts → Add Proxy Host):

Separador	Campo	Valor
Details	Domain Names	`waha.lan.ddx.pt`
Details	Scheme	`http`
Details	Forward Hostname / IP	`192.168.200.6`
Details	Forward Port	`3000`
Details	Cache Assets	desligado
Details	Block Common Exploits	ligado
Details	Websockets Support	**ligado** (necessário para `/ws`)
Details	Access List	**`LAN-only`**
SSL	SSL Certificate	Let's Encrypt com **DNS Challenge** (o HTTP challenge exige acesso público)
SSL	Force SSL / HTTP/2	ligado

Advanced (Custom Nginx Configuration):

```
client_max_body_size 50m;
proxy_read_timeout 300s;
proxy_send_timeout 300s;
```

DNS: crie o registo `waha.lan.ddx.pt` a apontar para `192.168.200.6` (IP privado) no DNS interno ou no DNS público. Mesmo que alguém de fora resolva o nome, o IP privado não é alcançável da internet e a Access List bloqueia qualquer origem fora da LAN.

> Atenção: a Access List baseia-se no IP de origem. Se o NPM estiver atrás de outro proxy (ex.: Cloudflare), o IP visto será o do proxy e a restrição deixa de ser fiável — nesse caso use apenas a opção 8.1.

8.3 Osteogest — ajustes no proxy host existente

Nos proxy hosts `demo.osteogest.ddx.pt` e `vilma.osteogest.ddx.pt`:

Details: manter Websockets Support ligado e Block Common Exploits ligado.

Advanced (Custom Nginx Configuration) — acrescentar:

```
# O webhook do WAHA chega pela rede Docker interna; bloquear o acesso público
location ^~ /api/webhooks/waha {
    return 404;
}

# Envio de PDFs/imagens aos pacientes
client_max_body_size 25m;
proxy_read_timeout 120s;
```

Isto garante que o endpoint de webhooks **nunca** é chamado a partir da internet (defesa em profundidade,

para além da validação HMAC no Osteogest).

9. Firewall (opcional, reforço)

O Docker insere as suas próprias regras de iptables e **ignora o UFW** para portas publicadas. O bind ao IP da LAN já impede o acesso externo; para restringir também quem, dentro da LAN, pode chegar à porta 3000:

```
# permitir apenas a sub-rede da LAN a chegar ao WAHA publicado
sudo iptables -I DOCKER-USER -p tcp --dport 3000 ! -s 192.168.200.0/24 -j DROP

# tornar persistente
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
```

Para reverter: `sudo iptables -D DOCKER-USER -p tcp --dport 3000 ! -s 192.168.200.0/24 -j DROP`.

10. Produção (vilma.osteogest.ddx.pt)

- ☐ Descobrir o IP LAN do servidor (`ip -4 addr`) e a sub-rede.
- ☐ Gerar **novos** segredos (secção 3).
- ☐ Criar a stack `waha` com `WAHA_LAN_IP` = IP LAN de produção.
- ☐ Fixar a versão da imagem (`WAHA_IMAGE_TAG=gows-<versão>`).
- ☐ Ligar a stack do Osteogest de produção à rede `waha_net`.
- ☐ Aplicar os ajustes do NPM (8.3) ao proxy host `vilma.osteogest.ddx.pt`.
- ☐ Validar (secção 6), incluindo o teste a partir de 4G.
- ☐ Emparelhar o número de produção da clínica.
- ☐ Configurar backups (secção 11).

11. Manutenção

11.1 Atualizar o WAHA

Portainer → Stacks → `waha` → **Pull and redeploy** (método Repository) ou **Update the stack** com Re-pull image ligado (Web editor). As sessões mantêm-se porque estão no volume `waha_sessions`.

11.2 Backups das sessões

Com `name: waha`, o Docker cria os volumes `waha_waha_sessions` e `waha_waha_media`.

```
# backup
docker run --rm -v waha_waha_sessions:/data -v "$PWD":/backup alpine \
  tar czf /backup/waha_sessions_$(date +%F).tgz -C /data .

# restauro (com a stack parada)
docker run --rm -v waha_waha_sessions:/data -v "$PWD":/backup alpine \
  sh -c "rm -rf /data/* && tar xzf /backup/waha_sessions_AAAA-MM-DD.tgz -C /data"
```

> Os ficheiros de sessão permitem enviar mensagens em nome do número da clínica. Guarde os backups cifrados e com acesso restrito.

11.3 Rodar segredos

1. Gerar novo valor.

2. Atualizar a variável nas **duas** stacks (WAHA e Osteogest).
3. Update the stack em ambas.

12. Resolução de problemas

Sintoma	Causa provável	Ação
`401 Unauthorized` na API	`X-Api-Key` errada ou em falta	Confirmar `WAHA_API_KEY` nas duas stacks
Osteogest não resolve `waha`	Osteogest fora da rede `waha_net`	Rever secção 5 e Update the stack
Stack não arranca: "define WAHA_..."	Variável obrigatória vazia	Preencher na secção Environment variables
Sessão em `SCAN_QR_CODE` após reinício	Volume de sessões perdido / número desligado no telemóvel	Voltar a emparelhar; verificar volume
Sessão `FAILED`	Ligação WhatsApp rejeitada	`POST /api/sessions/osteogest/restart`; ver `docker logs waha`
Webhooks não chegam	`OSTEOGEST_WEBHOOK_URL` errada ou rede	`docker logs waha
Webhook responde 401 no Osteogest	HMAC diferente entre stacks	Igualar `WAHA_HOOK_HMAC_KEY` / `WAHA_WEBHOOK_HMAC_KEY`
Container `unhealthy`	WAHA não responde em `/ping`	Ver logs; confirmar recursos (RAM/CPU)
Mensagem com erro 463	Restrição temporária da conta a novos contactos	Reduzir envios a números novos; aguardar

13. Boas práticas de utilização

- Enviar apenas a pacientes que deram **consentimento** para contacto por WhatsApp.
- Evitar envios em massa e mensagens a números que nunca contactaram a clínica (risco de bloqueio do número).
- Espaçar envios (alguns segundos entre mensagens).
- **Não** incluir diagnósticos, notas clínicas ou outros dados de saúde nas mensagens (RGPD).
- Monitorizar o estado da sessão (o Osteogest deve alertar quando deixar de estar `WORKING`).